Parte 1 - Escolha da ferramenta

Nome da ferramenta utilizada: Gemini 3.5 Flash Estendido
Data de uso: 08/06/2026
Versão paga
Possui acesso à internet

Parte 2 - Explicação conceitual

Prompt simples utilizado: "Explique o que são estruturas de dados"
Resposta ao prompt simples:

De forma simples, **estruturas de dados** são formas organizadas de armazenar, gerenciar e relacionar informações no computador para que possamos acessá-las e modificá-las de maneira eficiente.

Pense nelas como os organizadores do mundo digital. Se você guardar todas as suas roupas jogadas em uma grande pilha, vai demorar muito para encontrar uma meia específica. Mas se usar gavetas, cabides e colmeias organizadoras, o acesso se torna instantâneo. No código, a lógica é a mesma: dependendo do seu problema, você precisará de uma "gaveta" diferente.

Os Tipos Mais Comuns

As estruturas de dados são divididas entre **lineares** (elementos em sequência) e **não-lineares** (elementos conectados de forma mais complexa).

1. Estruturas Lineares

Os elementos estão dispostos um atrás do outro, em uma linha direta.

- **Vetor (Array):** Uma sequência de elementos armazenados em posições contíguas na memória. O acesso é super rápido através de um índice (ex: `lista[0]`), mas mudar o tamanho dele no meio do caminho costuma ser custoso.
- **Lista Encadeada (Linked List):** Cada elemento (nó) conhece o seu valor e aponta para a direção do próximo elemento na memória. É ótima para inserções e remoções dinâmicas, mas para achar o último item, você precisa percorrer a lista desde o início.
- **Pilha (Stack):** Funciona no modelo **LIFO** (*Last In, First Out* - O último que entra é o primeiro que sai). Pense em uma pilha de pratos: você sempre adiciona e remove do topo. É a estrutura por trás do botão "Desfazer" (Ctrl+Z) de qualquer software.
- **Fila (Queue):** Funciona no modelo **FIFO** (*First In, First Out* - O primeiro que entra é o primeiro que sai). Igual a uma fila de banco ou o envio de requisições para um servidor.

2. Estruturas Não-Lineares

Aqui os dados não seguem uma sequência única; eles se ramificam ou se conectam em redes.

- **Árvores (Trees):** Estruturas hierárquicas. Existe um nó raiz no topo que se ramifica em nós filhos. Muito usada em sistemas de arquivos (pastas e subpastas) e na renderização de interfaces (como a árvore de componentes/widgets de um app).
- **Grafos (Graphs):** Uma rede de nós (vértices) conectados por linhas (arestas). É a estrutura perfeita para mapear redes sociais (amigos conectados), rotas de GPS (cidades e estradas) ou sistemas de recomendação.

3. Tabelas Hash (Dicionários ou Mapas)

Uma das estruturas mais poderosas da computação. Ela associa uma **chave** a um **valor** (como um dicionário de verdade, onde você busca pela palavra para achar a definição). Através de uma função matemática (função hash), ela consegue encontrar qualquer dado quase instantaneamente, sem precisar rodar loops pela memória.

Por que isso importa na prática?

A escolha da estrutura de dados impacta diretamente a **performance** do software (tempo de execução e consumo de memória).

Se você precisa apenas buscar elementos pelo índice a todo momento, um Array resolve brilhantemente. Se o seu app precisa gerenciar o histórico de telas que o usuário navegou para conseguir voltar uma a uma, uma Pilha é a escolha natural.

Escrever um código eficiente é, em grande parte, saber casar o algoritmo certo com a estrutura de dados ideal para aquele cenário.

Qual dessas estruturas você costuma usar mais no seu dia a dia, ou tem mais curiosidade em ver como funciona por baixo dos panos?

Ferramenta utilizada: Gemini 3.5 Flash Estendido
Data: 08/06/2026
Trecho da resposta aproveitado: Quase a resposta inteira, exceto pela pergunta ao final
Como a resposta foi verificada: Pesquisas na internet para cruzar as principais informações relevantes apresentadas

---

Prompt elaborado utilizado:

> Contexto: Sou calouro em Ciência da Computação na UFRGS e tenho dúvidas quanto ao que são estruturas de dados, especialmente a definição mais formal e acadêmica.
> Público-Alvo: Um estudante de primeiro semestre de Ciência da Computação, com curso técnico no Ensino Médio em desenvolvimento web fullstack e um ano de experiência com desenvolvimento mobile, que aprende bem visualizando exemplos práticos.
> Formato Desejado: Um guia didático estruturado em Markdown, utilizando analogias do mundo real para explicar os 5 tipos mais comuns de estruturas de dados. Inclua uma seção curta explicando por que um programador precisa dominar isso.

Resposta ao prompt elaborado:

O Universo das Estruturas de Dados: Do Acadêmico ao Prático

Parabéns pelo ingresso na UFRGS! Deixar o ecossistema de desenvolvimento focado puramente em interfaces e APIs para mergulhar nos fundamentos da Ciência da Computação pode parecer um choque teórico no início, mas é exatamente aqui que a mágica da eficiência acontece.

Vamos traduzir esse conceito que separa quem apenas coda de quem projeta sistemas de alto desempenho.

---

1. A Definição Formal (O Lado Acadêmico)

Na academia, você ouvirá que algoritmos e estruturas de dados são duas faces da mesma moeda. Enquanto o algoritmo é a sequência de passos para resolver um problema, a **Estrutura de Dados** é a forma como organizamos as informações na memória para que esse algoritmo rode de forma eficiente.

Formalmente, podemos definir uma estrutura de dados como:

> Uma coleção de valores de dados, os relacionamentos entre eles e as funções ou operações que podem ser aplicadas a esses dados.

Em termos matemáticos mais estritos, muitas vezes modelamos uma estrutura como um **Tipo Abstrato de Dados (TAD)**, definido por uma tupla:

$$TAD = (V, O)$$

T AD = (V , O)
Onde:

- VV é o conjunto de valores (o domínio dos dados e as relações lógicas entre eles).
- OO é o conjunto de operações (as regras de acesso, inserção e remoção) que preservam a integridade desse conjunto.

Em termos práticos para quem já programa: uma classe no seu código que gerencia coleções de objetos seguindo regras estritas de inserção e busca é uma implementação de uma estrutura de dados.

---

2. Por que você precisa dominar isso?

Se você está construindo uma aplicação mobile ou uma API simples, o próprio framework costuma abstrair a memória para você. Mas pense no seguinte:

- **Escalabilidade:** Um loop simples em uma lista com 10 elementos roda instantaneamente. Mas e se a base de usuários do seu sistema crescer para 10 milhões? A estrutura de dados errada pode transformar um milissegundo de espera em minutos de travamento.
- **Gestão de Recursos:** Dispositivos móveis e servidores possuem limites de memória e processamento. Escolher a estrutura certa minimiza o consumo de bateria e RAM.
- **Escrever Código Limpo e Semântico:** Quando você entende a estrutura ideal para o problema, você para de fazer gambiarras com loops aninhados e resolve o problema na raiz da modelagem de dados.

---

3. As 5 Estruturas de Dados Mais Comuns

Para facilitar a visualização, imagine a memória do computador como um grande armazém. Cada estrutura organiza as caixas nesse armazém de um jeito diferente.

A. Vetores (Arrays / Lists)

É a estrutura mais fundamental. Os dados são guardados de forma **contígua** (colados uns nos outros) na memória, e cada elemento tem uma posição fixa chamada **índice** (começando do zero).

- **A Analogia:** Pense em um armário organizador com gavetas numeradas de 0 a 9. Se você sabe o número da gaveta, abre e pega o objeto instantaneamente.
- **No mundo real:** A lista de itens que você renderiza em uma tela de feed.
- **Vantagem:** Acesso direto e ultra-rápido a qualquer elemento via índice.
- **Desvantagem:** Mudar o tamanho do vetor é custoso. Se o armário estiver cheio e você precisar de mais uma gaveta, terá que comprar um armário maior e passar tudo para ele.

B. Pilhas (Stacks)

Uma estrutura linear onde o acesso é restrito: você só interage com o elemento que está no topo. Segue a filosofia **LIFO** (*Last In, First Out* - O último que entra é o primeiro que sai).

- **A Analogia:** Uma pilha de pratos na cozinha. Você sempre coloca o novo prato limpo no topo. Se precisar pegar um prato, vai tirar o do topo primeiro. Tentar puxar o prato da base vai quebrar tudo.
- **No mundo real:** O botão "Voltar" (a navegação de telas do celular empilha as telas abertas; quando você volta, ele remove a do topo) ou a função de "Desfazer" (`Ctrl + Z`) do seu editor de código.

C. Filas (Queues)

Ao contrário da pilha, a fila segue a filosofia **FIFO** (*First In, First Out* - O primeiro que entra é o primeiro que sai). A inserção acontece no final e a remoção acontece no início.

- **A Analogia:** A tradicional e inevitável fila do RU (Restaurante Universitário). Quem chega primeiro é atendido primeiro e sai da fila. Quem chega depois vai para o final.
- **No mundo real:** O gerenciamento de requisições de rede chegando em um servidor, ou o envio de notificações push em lote para os dispositivos.

D. Listas Encadeadas (Linked Lists)

Diferente do Array, os elementos aqui não estão juntos na memória. Eles podem estar espalhados em qualquer canto. Cada elemento (chamado de **Nó**) guarda duas coisas: o dado em si e um **ponteiro** (uma seta) indicando o endereço de memória do próximo elemento.

- **A Analogia:** Uma caça ao tesouro. Você recebe uma pista na largada. Essa pista não é o tesouro, mas te diz exatamente onde ir para achar o próximo bilhete. O jogo só acaba quando um bilhete diz "Fim".
- **No mundo real:** A implementação do histórico de navegação de "Avançar" e "Voltar" de um navegador web.
- **Vantagem:** É muito fácil inserir ou remover elementos no meio da lista; basta mudar a "seta" de direção dos nós vizinhos, sem precisar mover o resto dos dados de lugar.

E. Árvores (Trees)

Uma estrutura **não-linear** e hierárquica. Existe um nó principal no topo (chamado de **Raiz**) que se conecta a outros nós ("Filhos"). A mais famosa é a Árvore Binária de Busca, onde cada nó tem no máximo dois filhos (os menores ficam à esquerda, os maiores à direita).

- **A Analogia:** A estrutura de pastas do seu sistema operacional. Você tem a pasta raiz (`/`), dentro dela tem `Documentos` e `Downloads` , e dentro de `Documentos` tem subpastas.
- **No mundo real:** O DOM do navegador (a estrutura HTML), as árvores de componentes de interfaces mobile, ou sistemas de busca rápida em bancos de dados (como árvores B+).

Para consolidar essa transição para o ambiente acadêmico: qual dessas estruturas você acha que o seu framework ou linguagem de preferência utiliza por baixo dos panos quando você cria uma lista dinâmica?

Ferramenta utilizada: Gemini 3.5 Flash Estendido
Data: 08/06/2026
Trecho da resposta aproveitado: Quase a resposta inteira, exceto pelos dois primeiros parágrafos e pela pergunta ao final
Como a resposta foi verificada: Pesquisas na internet para cruzar as principais informações relevantes apresentadas
Comparação entre as duas respostas: A resposta ao prompt simples acabou sendo mais geral/abrangente e focando em iniciantes em tecnologia, dado que o prompt apenas pediu explicação de estruturas de dados sem nenhum contexto adicional, usou uma abordagem mais intuitiva e descritiva, com baixa profundidade teórica e analogias mais genéricas de fila de bannco, pratos na cozinha, gavetas de roupa etc., focando em entender o conceito de forma rápida e simples. A resposta ao prompt elaborado considerou meu background técnico, fazendo uma ponte entre o rigor matemático e acadêmico (provavelmente por ele saber que sou calouro da UFRGS) e a prática de engenharia de software (provavelmente por eu ter dito que trabalho na área). Em função disso, aprofundou mais no tema, introduzindo o conceito de Tipo Abstrato de Dados e a modelagem matemática TAD = (V, O), por exemplo. Também usou analogias mais específicas de fila do RU, caça ao tesouro e árvore DOM/de Componentes, focando mais em entender o porquê de estruturas de dados importarem (escala, consumo de RAM, arquitetura etc.). É importante notar que, por conta da forma como o Gemini funciona na web e lendo as respostas, é extremamente provável que a resposta ao prompt mais elaborado tenha considerado também o contexto pessoal que ele tem armazenado sobre mim de conversas prévias. Como o prompt menciona mais informações sobre minha vida e especifica uma persona igual a mim no público-alvo, ele julgou pertinente trazer informações de outros chats. No prompt mais simples, como a pergunta era estritamente objetiva, direta e simples, ele optou por não buscar esse contexto anterior, o que também influenciou na resposta.

Parte 3 - Segunda tarefa: geração de código

Prompt simples utilizado: "me dê um código de um jogo de adivinhação de números"
Resposta ao prompt simples utilizado:

Jogo de Adivinhação de Números em Python

Aqui está um código completo e direto em Python. O computador escolhe um número secreto entre 1 e 100, e o seu trabalho é tentar adivinhar. O programa te dá dicas se o número correto é maior ou menor a cada tentativa.

```python
import random

def jogo_adivinhacao():
    print("========================================")
    print("🎯 Bem-vindo ao Jogo de Adivinhação! 🎯")
    print("========================================")
    print("Eu pensei em um número entre 1 e 100.")
    print("Será que você consegue adivinhar qual é?\n")

    # O computador escolhe um número aleatório entre 1 e 100
    numero_secreto = random.randint(1, 100)
    tentativas = 0
    acertou = False

    while not acertou:
        try:
            # Pede o palpite do jogador
            palpite = int(input("Digite o seu palpite: "))
            tentativas += 1

            # Validação para garantir que o número está no intervalo correto
            if palpite < 1 or palpite > 100:
                print("⚠️ Ei! O número está entre 1 e 100. Foco no intervalo!")
                continue

            # Verifica o palpite
            if palpite == numero_secreto:
                acertou = True
                print(f"\n🎉 Parabéns! Você acertou o número {numero_secreto} em {tentativas} tenta
            elif palpite < numero_secreto:
                print("📉 Errou... O número secreto é MAIOR.")
            else:
                print("📈 Errou... O número secreto é MENOR.")

        except ValueError:
            print("❌ Por favor, digite apenas números inteiros válidos.")

    print("\nObrigado por jogar! Até a próxima. 👋")

# Executa o jogo
if __name__ == "__main__":
    jogo_adivinhacao()
```

🛠️ Como o código funciona?

- `import random` : Importa a biblioteca nativa do Python usada para gerar o número aleatório.
- `random.randint(1, 100)` : Sorteia o número secreto que você precisa adivinhar (incluindo o 1 e o 100).
- `while not acertou` : Cria um loop que continua rodando e pedindo palpites até que você acerte.
- 💥 **Tratamento de Erros (`try/except`)**: Se você digitar uma letra ou um caractere especial sem querer, o programa não vai "quebrar". Ele avisa que o input é inválido e te dá outra chance.

Para rodar, basta copiar o código acima, salvar em um arquivo como `jogo.py` e executá-lo no seu terminal ou na sua IDE favorita.

Código gerado (extraído da resposta da IA acima):

```python
import random

def jogo_adivinhacao():
    print("========================================")
    print("🎯 Bem-vindo ao Jogo de Adivinhação! 🎯")
    print("========================================")
    print("Eu pensei em um número entre 1 e 100.")
    print("Será que você consegue adivinhar qual é?\n")

    # O computador escolhe um número aleatório entre 1 e 100
    numero_secreto = random.randint(1, 100)
    tentativas = 0
    acertou = False

    while not acertou:
        try:
            # Pede o palpite do jogador
            palpite = int(input("Digite o seu palpite: "))
            tentativas += 1

            # Validação para garantir que o número está no intervalo correto
            if palpite < 1 or palpite > 100:
                print("⚠️ Ei! O número está entre 1 e 100. Foco no intervalo!")
                continue

            # Verifica o palpite
            if palpite == numero_secreto:
                acertou = True
                print(f"\n🎉 Parabéns! Você acertou o número {numero_secreto} em {tentativas} tentativa
            elif palpite < numero_secreto:
                print("📉 Errou... O número secreto é MAIOR.")
            else:
                print("📈 Errou... O número secreto é MENOR.")

        except ValueError:
            print("❌ Por favor, digite apenas números inteiros válidos.")

    print("\nObrigado por jogar! Até a próxima. 👋")

# Executa o jogo
if __name__ == "__main__":
    jogo_adivinhacao()
```

Ferramenta utilizada: Gemini 3.5 Flash Estendido
Data: 11/06/2026
Trecho da resposta aproveitado: Apenas o código fornecido
Como a resposta foi verificada: Análise do código com base em conhecimentos prévios e teste manual com exemplos de inputs válidos e inválidos
Explicação do código: ele gera aleatoriamente um número de 1 a 100 e pede para o usuário chutar um número. Então, ele indica se o número está acima ou abaixo do valor digitado e pede para o usuário digitar um novo valor, repetindo o processo até que ele acerte. Ao acertar, o jogo acaba e é exibido o número correto e quantas tentativas foram necessárias. Há validações de intervalo, caso o usuário digite números menores que 1 ou maiores que 100, e de formato, para caso o input não seja um número.

1º teste realizado:

```
========================================
🎯 Bem-vindo ao Jogo de Adivinhação! 🎯
========================================
Eu pensei em um número entre 1 e 100.
Será que você consegue adivinhar qual é?

Digite o seu palpite: 50
📈 Errou... O número secreto é MENOR.
Digite o seu palpite: 25
📈 Errou... O número secreto é MENOR.
Digite o seu palpite: 13
📈 Errou... O número secreto é MENOR.
Digite o seu palpite: 7
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 10
📈 Errou... O número secreto é MENOR.
Digite o seu palpite: 8
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 9

🎉 Parabéns! Você acertou o número 9 em 7 tentativas!

Obrigado por jogar! Até a próxima. 👋
```

2º teste realizado:

```
========================================
🎯 Bem-vindo ao Jogo de Adivinhação! 🎯
========================================
Eu pensei em um número entre 1 e 100.
Será que você consegue adivinhar qual é?

Digite o seu palpite: batata
❌ Por favor, digite apenas números inteiros válidos.
Digite o seu palpite: 101
⚠️ Ei! O número está entre 1 e 100. Foco no intervalo!
Digite o seu palpite: 0
⚠️ Ei! O número está entre 1 e 100. Foco no intervalo!
Digite o seu palpite: 50
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 75
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 87
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 94
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 97
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 98
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 99
📉 Errou... O número secreto é MAIOR.
Digite o seu palpite: 100

🎉 Parabéns! Você acertou o número 100 em 10 tentativas!

Obrigado por jogar! Até a próxima. 👋
```

Análise crítica da solução produzida: o programa gerado é bem simples, mas há um erro notável em relação ao comportamento ideal. Se eu digito um valor inválido, como um texto ou um número fora do intervalo, ele soma o valor à quantidade de tentativas mesmo assim. O que não faz sentido, visto que meu input foi recusado por ser inválido. Na minha concepção, isso não deveria somar às tentativas. De resto, o programa funciona de forma muito semelhante a como eu faria manualmente.

Parte 4 - Melhorando o prompt

Prompt elaborado utilizado:

> Escreva o código de um jogo de adivinhação de números nos seguintes critérios:
> Linguagem de programação: Python.
> Nível de explicação: Forneça uma explicação passo a passo da lógica utilizada, adequada para um estudante universitário de computação.
> Comentários: Inclua comentários curtos e diretos no próprio código para explicar o que cada bloco lógico (variáveis, laços e condicionais) está fazendo.
> Exemplos de entrada e saída: Após o código, mostre um bloco de texto simulando uma partida no console, evidenciando as entradas do usuário e as respostas esperadas do sistema (ex: "Muito alto", "Muito baixo", "Você acertou").
> Tratamento de erros: O código deve validar a entrada do usuário, garantindo que o programa não quebre caso ele digite letras, caracteres especiais ou envie um texto em branco, solicitando que ele digite novamente.
> Restrições de implementação: O número a ser adivinhado deve estar entre 1 e 100. Utilize um laço while para manter o jogo rodando até o usuário acertar, e não utilize nenhuma biblioteca de terceiros, apenas as nativas da linguagem.

Resposta ao prompt elaborado utilizado:

Aqui está a implementação do jogo de adivinhação em Python, projetado seguindo as boas práticas de desenvolvimento, controle de fluxo e tratamento de exceções.

```python
import random


def jogar_adivinhacao():
    # Inicializa o número secreto aleatório entre 1 e 100
    numero_secreto = random.randint(1, 100)
    tentativas = 0

    print("=== Bem-vindo ao Jogo de Adivinhação! ===")
    print("Eu pensei em um número entre 1 e 100. Tente adivinhar!")

    # Laço principal do jogo: roda até que o usuário acerte o número
    while True:
        entrada = input("\nDigite o seu palpite: ").strip()

        # Tratamento de erro: valida se a entrada está vazia ou não é numérica
        try:
            # Tenta converter a entrada para inteiro
            palpite = int(entrada)
        except ValueError:
            # Captura o erro caso a conversão falhe (letras, caracteres especiais ou vazio)
            print(
                "Entrada inválida! Por favor, digite apenas números inteiros."
            )
            continue

        # Validação de intervalo: garante que o número está entre 1 e 100
        if palpite < 1 or palpite > 100:
            print("Palpite fora do intervalo! O número está entre 1 e 100.")
            continue

        # Incrementa o contador de tentativas válidas
        tentativas += 1

        # Bloco condicional para verificar o palpite do usuário
        if palpite < numero_secreto:
            print("Muito baixo! Tente um número maior.")
        elif palpite > numero_secreto:
            print("Muito alto! Tente um número menor.")
        else:
            # Condição de acerto: encerra o laço principal
            print(
                f"\nParabéns! Você acertou o número {numero_secreto} em {tentativas} tentativas!"
            )
            break


# Garante que o jogo execute apenas se o arquivo for rodado diretamente
if __name__ == "__main__":
    jogar_adivinhacao()
```

Explicação Lógica Passo a Passo

Para um estudante de computação, a estrutura deste programa demonstra conceitos fundamentais de robustez de código, manipulação de estado e controle de fluxo.

1. **Importação e Geração de Estado (`random.randint`)**: Utiliza-se a biblioteca nativa `random` para gerar um pseudoaleatório no intervalo fechado $[1, 100]$[1, 100]. A variável `numero_secreto` guarda o estado estático que o jogador deve alcançar, enquanto `tentativas` atua como um acumulador.
2. **Laço de Repetição Indeterminado (`while True`)**: Como não sabemos quantas vezes o usuário vai errar, um laço `while` infinito (controlado manualmente) é a estrutura ideal. O fluxo só é interrompido quando a palavra-chave `break` é atingida no bloco de acerto.
3. **Tratamento de Exceções (`try-except`)**: Em aplicações reais, a entrada de dados do usuário é uma das maiores fontes de falhas estruturais. O método `int()` lança uma exceção do tipo `ValueError` se receber strings literais, caracteres especiais ou espaços em branco. O bloco `try-except` captura essa falha antes que ela cause o crash do interpretador, exibindo uma mensagem amigável.
4. **Palavra-chave `continue**` : Caso a entrada seja inválida ou esteja fora do escopo estipulado ($1$1 a 100 100), o `continue` aborta a iteração atual do laço e força o programa a voltar para o topo do `while` , ignorando as verificações de maior/menor e a contagem de tentativas.
5. **Estrutura Condicional (`if / elif / else`)**: Uma árvore de decisão simples avalia a relação de ordem entre o palpite e o número secreto. Por eliminação lógica, se o palpite não é menor (`if`) e não é maior (`elif`), ele obrigatoriamente é igual, acionando o `else` que finaliza o programa.

Exemplo de Entrada e Saída no Console

Abaixo está uma simulação de como o programa se comporta no terminal, demonstrando o tratamento de erros e o fluxo das dicas:

```
=== Bem-vindo ao Jogo de Adivinhação! ===
Eu pensei em um número entre 1 e 100. Tente adivinhar!

Digite o seu palpite:
Entrada inválida! Por favor, digite apenas números inteiros.

Digite o seu palpite: texto123
Entrada inválida! Por favor, digite apenas números inteiros.

Digite o seu palpite: 150
Palpite fora do intervalo! O número está entre 1 e 100.

Digite o seu palpite: 50
Muito baixo! Tente um número maior.

Digite o seu palpite: 75
Muito alto! Tente um número menor.

Digite o seu palpite: 62
Muito baixo! Tente um número maior.

Digite o seu palpite: 68
Parabéns! Você acertou o número 68 em 4 tentativas!
```

Ferramenta utilizada: Gemini 3.5 Flash Estendido
Data: 13/06/2026
Trecho da resposta aproveitado: Quase a resposta inteira, exceto pelo primeiro parágrafo, mas principalmente o código, a explicação e os testes
Como a resposta foi verificada: Análise da resposta com base em conhecimentos prévios
Comparação entre as duas respostas: A resposta ao prompt elaborado foi levemente mais adequada. O prompt simples deu um código quase integralmente funcional, mas com um errinho na lógica de contagem de tentativas, e o complexo corrigiu esse problema colocando o "tentativas += 1" depois das validações de entrada. Essa segunda resposta também colocou explicações da lógica que facilitam entender o funcionamento, além de comentários no código, exemplos de entrada e saída, tratamento de erros etc. mais completos. Entretanto, mesmo com o prompt mais elaborado, ele usou uma estrutura que normalmente não é desejável com "while True" e "break" em vez de "while not acertou", que acaba sendo um pouco menos intuitivo e fugindo de padrões que preferem o uso de flags para esse tipo de controle. Imagino que a resposta ao prompt mais elaborado não foi tão significativamente superior porque o programa exigido originalmente já era extremamente simples, então não tem muito como ele inventar moda para dar um resultado notavelmente superior, fora que o modelo utilizado (O Gemini Flash 3.5 com pensamento estendido) é um dos melhores do mercado e já tende a dar respostas bem robustas mesmo sem muitas especificações.

Parte 5 - Verificação crítica

A resposta escolhida para a verificação foi a do prompt simples de geração de código da parte 3, porque ele tem um erro na contagem de tentativas.

1. **A resposta estava correta?** Quase integralmente, pois o código funciona e cumpre o objetivo principal do jogo, mas possui um erro de lógica na contagem de tentativas.
2. **Havia informações incompletas ou incorretas?** Sim. Tinha erro na lógica de incremento de tentativas. O contador é incrementado antes da validação do intervalo, o que significa que entradas inválidas e com formato errado são contabilizadas como tentativas, apesar de não serem palpites de fato.
3. **Como a resposta foi verificada?** Executando o código e testando inputs inválidos de propósito. Vendo a saída, deu para perceber que o contador de tentativas aumentava mesmo quando o programa não aceitava a entrada. Então olhei no código e vi que na linha 19 o "tentativas += 1" aparece antes da validação "if palpite < 1 or palpite > 100", o que não deveria acontecer.
4. **A LLM demonstrou excesso de confiança?** Sim. A LLM apresentou o código como "completo e direto" e explicou o funcionamento sem mencionar nenhum potencial problema.
5. **Como o prompt poderia ser melhorado?** O prompt poderia solicitar explicitamente que a contagem de tentativas considerasse apenas entradas válidas, uma revisão da lógica antes de entregar a resposta e um teste com inputs inválidos para verificar se não há nenhum problema, para então ser cruzado executando o código manualmente e vendo se as respostas batem com o esperado.

Parte 6 - Uso responsável de LLMs

1. **Por que não devemos copiar respostas sem revisão?** Porque LLMs podem gerar respostas erradas ou enviesadas.
2. **Por que é importante informar os prompts utilizados?** Porque ajuda a entender como certas informações foram obtidas ou códigos foram gerados, o que dá mais transparência e facilita possíveis futuras análises por parte de outras pessoas ou do próprio autor.
3. **Em quais situações as LLMs ajudam no aprendizado?** Para explicar conceitos abstratos de forma mais didática, gerar códigos de exemplo, revisar textos mostrando possíveis erros e sugerir melhorias a serem validadas pelo estudante.
4. **Em quais situações elas podem atrapalhar?** Quando o estudante coloca a IA para resolver as atividades por ele, quando ele não revisa as respostas, quando tem alucinações e referências bibliográficas que o aluno não valida... basicamente, quando o aluno só terceiriza as responsabilidades acadêmicas para a LLM.
5. **O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?** Significa usá-la para ajudar a encontrar soluções ou aprender conceitos, mas ter a responsabilidade intelectual de analisar, testar, corrigir e contextualizar as respostas, de modo a efetivamente compreender o tema.

Reflexão sobre o uso de IA Generativa

1. **O que você aprendeu sobre interação com LLMs?** Que é preciso ter muito cuidado, revisar tudo que a LLM gera e fazer prompts bem precisos e completos para obter bons resultados, incluindo, por exemplo, contexto, público-alvo e restrições.
2. **Como a qualidade do prompt influenciou os resultados?** Influenciou porque prompts mais detalhados propiciaram respostas mais especificamente adequadas para mim como usuário, códigos com menos erros, explicações mais detalhadas etc., enquanto prompts simples geraram explicações e códigos mais genéricos e com pequenos erros. Ou seja, a qualidade do prompt é o principal determinante da qualidade dos resultados.
3. **Em quais situações a ferramenta foi mais útil?** Principalmente na explicação dos conceitos de forma melhor direcionada para eu entender sem perder profundidade, mas levando meu background em consideração, e na geração do código adequado do jogo de adivinhação de números, que eu certamente demoraria mais do que a LLM caso fosse fazer manualmente.
4. **Em quais situações foi necessário verificar ou corrigir respostas?** Foi necessário na geração de código (Parte 3 e 4), onde a primeira versão tinha um bug de contagem. Também na Parte 2, onde a resposta ao prompt simples era mais genérica e exigia que eu filtrasse a informação mais relevante ao contexto acadêmico.
5. **Você utilizaria essa ferramenta em disciplinas futuras? Por quê?** Sim, mas com cuidado de revisão. Eu usaria para ajudar a entender conceitos confusos de forma mais didática e personalizada para mim e para gerar códigos mais rapidamente, porque isso permite melhorar minha compreensão dos conteúdos e poupar tempo de desenvolvimento.